

Individual Notebook Findings Report

Executive Summary

The **MM-IMDb** dataset (IMDb Multimodal Genre Classification) contains over 25,000 movies, each represented by a poster image and textual synopsis, with up to 23–27 multi-label genre tags ¹ ². This report maps a comprehensive analysis of this dataset into the sections of a standard notebook findings template. Key dataset facts (sources, variables, sample size, time range, types) and data-quality issues are summarized, major findings are extracted with supporting evidence, and each is aligned to the template's sections. Unspecified items (not present in the source analysis) are explicitly marked [UNSPECIFIED]. We propose additional analyses and visualizations (with example Python code) to fill gaps, and recommend a reproducible notebook workflow. A mapping table and a gap analysis table clarify which original report items correspond to which template sections and what actions are needed. We conclude with mermaid diagrams of the notebook flow, a deliverables timeline, and a prioritized checklist of tasks with time estimates.

Below is a mapping of original report content to the notebook template, followed by a gap analysis:

Original Report Content	Notebook Template Section
Dataset name, domain, size	1. Notebook Overview
Source (Kaggle dataset)	1. Notebook Overview
Problem type (multi-label classification)	1. Notebook Overview
Data quality issues (missing, duplicates)	2.1 Data Preprocessing
Feature engineering suggestions	2.1 Data Preprocessing
EDA results (distributions, correlations)	2.2 Exploratory Data Analysis (EDA)
Modeling suggestions (CLIP, BERT, baselines)	3.1 Model Selection
Model architectures recommended	3.2 Model Architecture/Structure
Training strategy (splits, epochs)	3.3 Training Process
Evaluation metrics (F1, precision, recall)	4.1 Metrics Used
Baseline performance targets (F1)	4.2 Performance Summary
Insights (multimodal utility, etc.)	6. Key Insights
Practical use cases (recommendations)	7. Practical Implications
Future work ideas	8. Future Work Suggestions

The gap analysis highlights where the notebook requires additional content or actual results:

Template Section	Covered by Report?	Gap / Action Required
1. Notebook Overview	Partial (dataset info)	Fill in title/author/date; add problem context
2.1 Data Preprocessing	Partial	Add code for missing-data handling, deduping
2.2 EDA	Partial	Produce actual plots (histograms, bar charts)
3.1 Model Selection	No	Specify chosen models (e.g. Logistic, CNN, CLIP)
3.2 Architecture/Structure	No	Detail network layers (BERT, ResNet, etc.)
3.3 Training Process	No	Document splits, epochs, hardware
4.1 Metrics Used	Partial (metrics listed)	Show code for computing metrics
4.2 Performance Summary	Partial (targets given)	Report actual results in tables/plots
5. Main Result	No	Summarize final model outcome (TBD)
6. Key Insights	Partial	Distill into bullets (see below)
7. Practical Implications	Partial	Expand on industry impact
8. Future Work Suggestions	Yes	(Original ideas presented)
9. Technical Details	No	Add environment specs and runtime info
10. Questions	No	List open research questions

1. Notebook Overview

Dataset: The “IMDb Multimodal Vision & NLP Genre Classification” (MM-IMDb) dataset from Kaggle ¹. It contains ~**25,000 movies** with both poster images and text (title, synopsis) ¹ ². Each movie can have multiple genres (up to 23–27 classes) ¹.

Domain & Problem: Multimodal (vision + NLP) multi-label classification of movie genres.

Sources: Data originates from IMDb/Kaggle (images in `poster` directory, tabular CSV with `overview`, `genres`, etc.).

Features: Key columns include `movie_id`, `title` (text), `overview` (text), `poster` (image file), `genres` (multi-label target), and `release_year` (numeric). (No extensive date/time or audio fields.)

Size & Range: ~25K samples. Release years range ~1900–2025 (mean ~2005 **【analysis】**). Genre count ~20 distinct labels (multi-label).

Data Types: Mix of text (synopsis), image (poster), categorical (genres, title), numeric (year).

Initial Preprocessing Mentioned: Identified issues include ~5–15% missing values (mostly in `overview` or missing posters), ~1–3% duplicate movies, and inconsistent label formats **【analysis】**. These inform the **Data Preprocessing** plan below.

2. Methodology Analysis

2.1 Data Preprocessing

- **Missing Data:** Identify columns with missing values:

```
missing_pct = df.isnull().mean()*100
print("Missing (%) per column:\n", missing_pct)
```

Report suggests ~5–15% missing, mainly in `overview` and some poster files. Strategy: drop or impute these rows. E.g.:

```
df = df.dropna(subset=['overview', 'poster'])
```

- **Duplicates:** Remove duplicate movie entries (e.g. same title/year):

```
dupe_count = df.duplicated(subset=['title', 'release_year']).sum()
df = df.drop_duplicates(subset=['title', 'release_year'])
```

- **Text Processing:** Tokenize synopses (e.g. with NLTK or spaCy), remove stop words, lemmatize. Convert to numerical features (TF-IDF or BERT embeddings).
- **Image Processing:** Load posters (with PIL/OpenCV), resize to uniform size, normalize pixel values (0–1).
- **Feature Engineering:**
- **Genres:** Convert the multi-label `genres` column into binary indicators using `MultiLabelBinarizer`. Example:

```
from sklearn.preprocessing import MultiLabelBinarizer
mlb = MultiLabelBinarizer()
y = mlb.fit_transform(df['genres'].str.split('|'))
```

- **Date/Year:** Scale or normalize `release_year`. E.g.:

```
from sklearn.preprocessing import StandardScaler
df['year_scaled'] = StandardScaler().fit_transform(df[['release_year']])
```

- **Other features:** Titles or other metadata could be encoded, but primary features are text/image.
- **Feature Selection/Reduction:** (If needed) apply PCA or truncate embeddings to reduce dimensionality after vectorization (especially for TF-IDF or image features).

2.2 Exploratory Data Analysis (EDA)

- **Visualization Techniques:** Use histograms, bar plots, scatter plots, and word clouds. For example, to view year distribution and genre counts:

```
import seaborn as sns
sns.histplot(df['release_year'], bins=20)
plt.title("Distribution of Release Years")

genre_counts = df['genres'].str.split('|').explode().value_counts()
sns.barplot(x=genre_counts.values, y=genre_counts.index)
plt.title("Genre Frequency")
```

- **Text Insights:** Plot a word cloud of all `overview` text to see common terms by genre, or bar plots of top keywords.
- **Numeric Analysis:** Summary stats (`df.describe()`), and check correlation between features (likely low between year and text).
- **Multimodal Relationship:** While harder to plot directly, we can do pairwise comparisons (e.g., representative posters vs. genres). Check if certain colors or aspects of images align with genre (horror posters are darker, etc.).
- **Data Quality Checks:** Visualize missing-data pattern (e.g. heatmap of nulls) and inspect duplicates. Summary of missing (%), duplicate count noted above.

3. Model Implementation

We outline candidate models and how to implement them.

3.1 Model Selection

- **Text-Only Baseline:** Logistic Regression or SVM on TF-IDF vectors of synopses. Multi-label via one-vs-rest.
- **Image-Only Baseline:** CNN (e.g. pre-trained ResNet50) on poster images with sigmoid outputs for genres.
- **Multimodal Models:**
 - *Concatenated Embedding Model:* Use pre-trained BERT (for text) and pre-trained ResNet (for images), concatenate their embeddings and train a top classifier.
 - *End-to-End CLIP:* A model like CLIP pre-trained on image-text can be fine-tuned for genre tags.
 - *Dual Network:* A small custom CNN + RNN or Transformer that jointly ingests image and text features.
- **Alternate Models:** Ensembles (e.g., average predictions of text and image models).
- **Libraries:** For example, in Python:

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression(solver='saga')
```

(Use PyTorch/TensorFlow for deep models.)

3.2 Model Architecture/Structure

- **Text model:** BERT-base (12-layer Transformer) fine-tuned with a multi-label output layer (sigmoid for each genre).
- **Image model:** ResNet50 or EfficientNet pretrained on ImageNet, with final layer replaced by sigmoid outputs for genre probabilities.
- **Fusion model:** Example structure:
 - Input text → BERT → 768-dim vector.
 - Input image → ResNet50 → 2048-dim vector.
 - Concatenate → Dense layers (e.g. 512→256) → Output layer (sigmoids for 23 genres).
- **Hyperparameters:** Learning rate (1e-5 for Transformers, 1e-4 for CNN), batch size (32), dropout (0.3), epochs (5–10).
- **Alternative:** Use a multimodal transformer (e.g. CLIP architecture) with shared vision-text attention.

3.3 Training Process

- **Data Split:** e.g. 70% train, 15% validation, 15% test (stratified on genres).
- **Cross-Validation:** 5-fold CV on train set for hyperparameter tuning.
- **Training:** Use GPUs (e.g. NVIDIA RTX 3090) due to image processing. Track training/val loss per epoch. Set a seed for reproducibility.
- **Example:**

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test =
train_test_split(df[['overview', 'poster']], y,
                  test_size=0.2,
                  random_state=42)
```

- **Monitoring:** Plot loss/accuracy vs. epochs. Use early stopping if validation loss increases.
- **Compute:** Record wall-clock time (likely hours). Ensure code logs final training and validation metrics.

4. Evaluation Metrics & Results

4.1 Metrics Used

- **Primary:** Multi-label **F1-score** (micro-averaged or macro-averaged) is most important. It balances precision and recall across genres.
- **Secondary:** Precision, Recall per genre, exact match ratio (all genres correct), and overall accuracy for reference.
- **Business Metric:** For recommendations, precision of top recommendations or “hit rate” may matter (not computed here).
- **Calculation Example:**

```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred, target_names=mlb.classes_))
```

4.2 Performance Summary

- **Benchmarks:** The original report suggests baselines: random ~5-10%, simple ~50-60% F1, advanced ~70-85% F1 **【analysis】** .
- **Results:** (UNSPECIFIED – actual numbers must come from running the models). We would fill in a table of precision/recall/F1 by genre. A bar chart of macro F1 scores per model and a confusion-matrix heatmap (for multi-label, e.g. showing co-occurrence) are useful.
- **Example:**

Model	F1-micro	Precision	Recall	----- ----- ----- -----				TF-IDF + Logistic
ResNet50 (images)	0.60	0.62	0.59	0.55	0.57	0.54		BERT + ResNet (fusion)
	0.75	0.77	0.74					

(Numbers illustrative – fill with actual scores.)

5. Main Result

The **primary outcome** of this analysis would be the best model's performance (e.g. "Our BERT+CNN fusion achieved X% F1"), but the report lacks experimental results. *[UNSPECIFIED]* denotes this unknown. In a complete notebook, we would present the chosen model's key metrics (F1, precision, recall) and a concise statement of improvement over baselines.

6. Key Insights

- Combining textual synopses and poster images yields stronger predictions than either alone. (Multimodal signals are complementary.)
- A significant proportion of synopses/images were missing or duplicate, so data cleaning is critical.
- Genre labels are **imbalanced**: a few genres dominate (Action, Drama) while others (Western, Musical) are rare, which can bias the model.
- Synopses often contain clear genre cues (keywords), making NLP very effective; images add context (e.g. color palettes, composition).
- Multi-label nature means some genres co-occur often (e.g. Action+Adventure), so modeling should account for label correlations.

7. Practical Implications

Improving genre classification helps **recommendation systems** and automated tagging. For example, better genre labels can enhance a movie recommendation engine by more accurately matching user tastes ³. In production, such a multimodal model could auto-tag new movies or cluster similar films. This analysis also demonstrates best practices in multimodal AI, which is valuable for portfolio and industry demonstrations.

8. Future Work Suggestions

- **Additional Modalities:** Incorporate audio (trailers), cast/crew data, or viewer reviews for tri-modal analysis ⁴.
- **Data Augmentation:** Expand dataset (more IMDb IDs) or augment images (flips, crops).
- **Refined Labels:** Relabel or merge ambiguous genre tags (e.g. standardize “Sci-Fi”/“Science Fiction”); consider hierarchical genres.
- **Advanced Models:** Experiment with CLIP or Vision-Language Transformers (e.g. ViLT).
- **Reproducibility:** Package the pipeline (using Docker or papermill) so others can run the notebook end-to-end.

9. Technical Details

- **Environment:** Python 3.x (e.g. 3.10), with libraries: `pandas`, `numpy`, `scikit-learn`, `matplotlib` / `seaborn`, `tensorflow` or `torch`, `Pillow` / `opencv-python`, `nltk` or `spacy`, etc.
- **Hardware:** GPU (e.g. NVIDIA RTX 3080/3090) recommended for training CNNs.
- **Runtime:** Training may take several hours per model. Software versions should be logged (e.g. `pip freeze`). For example, `TensorFlow 2.11` or `PyTorch 2.x`.

10. Questions for Further Investigation

- How much does each modality (text vs. image) contribute to accuracy? (Evaluate single-modality vs. fusion.)
- What is the optimal strategy for class imbalance (oversampling, focal loss, etc.) in this context?
- Can we improve performance by using pretraining on related data (e.g. movie review text or generic image-text datasets)?
- Would adding audio (movie trailers) or contextual metadata (director, cast) significantly improve predictions?
- In practice, should we optimize for precision (user trust) vs. recall (completeness) of genres?

Notebook Workflow Diagram

The notebook could follow this workflow (data → analysis → modeling → results):

```
graph LR
  A[Load & Clean Data] --> B[Exploratory Data Analysis]
  B --> C[Feature Engineering & Preprocessing]
  C --> D[Baseline Modeling (Text/CNN)]
  D --> E[Advanced Modeling (Multimodal)]
  E --> F[Evaluation & Metrics]
  F --> G[Conclusions & Insights]
```

Project Timeline (Gantt Chart)

```
gantt
    title Notebook Work Plan
    dateFormat YYYY-MM-DD
    section Tasks
    Data Loading & Cleaning      :done, 2026-04-30, 3d
    Exploratory Analysis        :active, 2026-05-03, 4d
    Modeling (Baseline)         : 2026-05-07, 4d
    Modeling (Advanced)         : 2026-05-11, 5d
    Evaluation & Documentation   : 2026-05-16, 3d
```

Prioritized Deliverables

Deliverable	Est. Effort	Dependencies
1. Data Load & Cleaning (code)	3 hrs	–
2. EDA & Visualization (plots, stats)	4 hrs	Task 1
3. Feature Engineering (text/image)	4 hrs	Task 1, 2
4. Baseline Models (train/eval)	4 hrs	Task 2, 3
5. Multimodal Model (develop & tune)	6 hrs	Task 4
6. Evaluation & Results (metrics, plots)	3 hrs	Task 4, 5
7. Notebook Documentation (write-up)	2 hrs	All above

Table: Deliverables with estimated effort (hours) and dependencies. Higher-priority tasks (data prep, EDA) are prerequisites for modeling.

Sources: Dataset description and multimodal context from Innovatiana ² ¹ ; recommended notebook and imports from GeeksforGeeks ⁵ ; and dataset use-cases from Innovatiana ³ ⁴ . Other content is inferred from the provided analysis report and common data-science practices.

¹ Knowledge Bridger: Towards Training-Free Missing Modality Completion
<https://arxiv.org/html/2502.19834v4>

² ³ ⁴ mm-IMDb Dataset: Multimodal Film Analysis | Innovatiana
<https://www.innovatiana.com/en/datasets/mm-imdb-multimodal-imdb-dataset>

⁵ Quick Guide to Exploratory Data Analysis Using Jupyter Notebook - GeeksforGeeks
<https://www.geeksforgeeks.org/data-analysis/quick-guide-to-exploratory-data-analysis-using-jupyter-notebook/>